

Advanced Parameter Passing

Department of Computer Science
University of Pretoria

6 May 2024

Previously when we discussed functions we considered

- return types, void and other simple types
- parameters, formal and actual, pass by value.
- separation of concerns

We will be considering functions and how to

- pass parameters by reference
- use `const` when passing parameters
- implement a simple recursive function

Up till now, you could only pass values to the function and not receive values from the function via the parameter list.

- Why would we want the calling code to receive updated values via the parameter list?
- What happens to the return type?
When do we use the return type?
- ...

Consider the 6.H in Study Unit 6 for swopping values, repeated below.

```
type value1 , value2 , temp;  
  
temp = value1;  
value1 = value2;  
value2 = temp;
```

Use this code to develop a swop function for swopping the two values. The function takes in two values (you may assume they are integers) and does not have a return type.

```
#include <iostream>

using namespace std;

// Place your swop function here

int main() {

    int x, y;

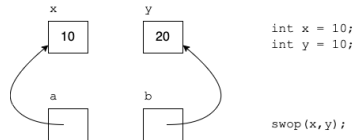
    x = 10;
    y = 20;

    swop(x,y);

    cout << "x = " << x << " and y = " << y << endl;

    return 0;
}
```

```
void swop(int &a, int &b) {  
    int temp;  
    temp = a;  
    a = b;  
    b = temp;  
}
```



- The reference (`&`) indicates pass by reference.
- `a` and `b` are referring to the same memory as what the variables calling the function are.
- The function must be called with variables, that is `swop(10,12)`; will not work.

- `constness` ensures the immutability of the parameter or variable. That means it cannot change.
- Consider the following example:

```
int power(const int base, int exponent) {  
    if (exponent == 0)  
        return 1;  
    return base * power(base, exponent - 1);  
}
```


There are two basic types of functions, iterative and recursive. The majority of the functions we write are iterative. The example we will write today is the Fibonacci sequence. In this case the iterative version is least computationally expensive.

What is the Fibonacci sequence?

It is a sequence of numbers,

- where the first two numbers are defined as being 1, and
- all subsequent numbers are the sum of the previous two numbers.

The sequence begins 1 1 2 3 5 8

<https://www.britannica.com/science/rational-number>

- Write a function implementing an iterative version of the Fibonacci sequence
- Identify the stop condition and the recursive call for the sequence
- Write a recursive implementation of the Fibonacci sequence
- Why was it stated that the iterative version is the least computationally expensive version of the sequence?

```
int fib(const int n) {  
    assert(n>0);  
    if ((1 == n) || (2 == n))  
        return 1;  
  
    int v1 = 1;  
    int v2 = 1;  
  
    for (int i = 2; i < n; i++) {  
        int result = v1 + v2;  
        v1 = v2;  
        v2 = result;  
    }  
  
    return v2;  
}
```

$$fib(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ fib(n-1) + fib(n-2) & \text{if } n > 2 \end{cases}$$

Identify the stop conditions and recursive call.

```
int fib(const int n) {  
    assert(n > 0);  
    if ((1 == n) || (2 == n))  
        return 1;  
    return fib(n-1) + fib(n-2);  
}
```

So why is the recursive version computationally expensive?